Building Tools for the DevOps Cycle

DevOps Practices and Principles — Study Notes

A concise guide to tools used across planning, development, integration, testing, release, deployment, operations, and monitoring.

1. What is the DevOps Cycle?

The DevOps cycle is a continuous process that brings software development (Development) and IT operations (Operations) together. The goal is to deliver software faster, more reliably, and with continuous feedback and improvement.

Stage	Purpose	Common Tools
Plan	Requirements, tasks, tracking	Jira, Trello, Azure Boards
Code	Write and manage source code	Git, GitHub, GitLab
Build	Compile/package application	Maven, Gradle, npm
Test	Automated/manual quality checks	JUnit, Selenium, pytest
Release	Prepare and manage releases	Jenkins, GitHub Actions
Deploy	Move application to environments	Docker, Kubernetes, Ansible
Operate	Run and maintain systems	Kubernetes, Ansible
Monitor	Observe performance and failures	Prometheus, Grafana, ELK

2. Source Code Management Tools

Source Code Management (SCM) tools store source code, maintain versions, and allow multiple developers to collaborate.

- Git: A distributed version-control system used to track changes in code.
- GitHub: A platform for hosting Git repositories and collaboration through pull requests, issues, and actions.
- GitLab: Provides Git repositories along with CI/CD, issue tracking, and DevOps features.
- Key concepts: repository, commit, branch, merge, pull request, clone, push, and pull.

3. Build Tools

Build tools automate compilation, dependency management, packaging, and preparation of software for testing or deployment.

- Maven: Popular Java build and dependency-management tool using a pom.xml file.
- Gradle: Flexible build automation tool commonly used with Java/Kotlin and Android projects.
- npm: Package manager and script runner widely used in JavaScript/Node.js projects.
- Typical flow: source code → dependencies → compile/build → package → artifact.

4. Continuous Integration and Continuous Delivery Tools

CI automatically builds and tests code whenever changes are integrated. Continuous Delivery/Deployment automates later stages so software can be released efficiently.

- Jenkins: Open-source automation server used to create CI/CD pipelines.
- GitHub Actions: Workflow automation integrated with GitHub repositories.
- GitLab CI/CD: Pipeline automation integrated into GitLab.
- Typical pipeline: checkout → install dependencies → build → test → package → deploy.

5. Testing Tools

Testing tools help detect defects early and verify that the application behaves as expected.

- JUnit: Unit-testing framework commonly used for Java applications.
- pytest: Python testing framework designed for simple and scalable tests.
- Selenium: Automates browser-based web application testing.
- Postman: Commonly used to test and validate APIs by sending HTTP requests.
- Testing types include unit, integration, system, regression, and acceptance testing.

6. Containerization Tools

Containerization packages an application together with its dependencies so it can run consistently across environments.

- Docker: Creates, runs, and manages containers.
- Dockerfile: Text file containing instructions for building a Docker image.
- Image: A packaged template used to create containers.
- Container: A running instance of an image.
- Benefits: portability, isolation, fast startup, and environment consistency.

7. Infrastructure as Code and Configuration Management

Infrastructure as Code (IaC) defines infrastructure using configuration files rather than manually creating resources.

- Ansible: Automates configuration, application deployment, and server management using playbooks.
- Terraform: Defines and provisions infrastructure using declarative configuration.
- Benefits: repeatability, automation, version control, and reduced configuration errors.

8. Orchestration Tools

When applications use many containers or services, orchestration tools automate scheduling, scaling, networking, and recovery.

- Kubernetes: Container orchestration platform for deploying, scaling, and managing containerized applications.
- Important concepts: cluster, node, pod, deployment, service, namespace, and config map.
- Kubernetes can restart failed containers and scale workloads according to requirements.

9. Monitoring and Logging Tools

Monitoring provides information about system health and performance. Logging records events that help developers troubleshoot failures.

- Prometheus: Collects and stores time-series metrics.
- Grafana: Creates dashboards and visualizations from monitoring data.
- ELK Stack: Elasticsearch, Logstash, and Kibana are commonly used for log collection, processing, searching, and visualization.
- Common metrics: CPU usage, memory, latency, request rate, error rate, and availability.

10. DevOps Toolchain

A DevOps toolchain combines specialized tools into an automated workflow. A sample toolchain is:

- Plan: Jira
- Code: Git + GitHub
- Build: Maven / Gradle / npm
- CI: Jenkins / GitHub Actions
- Test: JUnit / Selenium / pytest
- Containerize: Docker
- Deploy/Orchestrate: Kubernetes
- Monitor: Prometheus + Grafana
- Log analysis: ELK

11. Important Advantages of DevOps Tools

- Automation reduces repetitive manual work.
- CI/CD enables faster and more frequent software delivery.
- Version control provides traceability and collaboration.
- Containers provide consistent environments.
- Monitoring and logging help identify problems quickly.
- Infrastructure as Code makes environments reproducible.
- Continuous feedback supports continuous improvement.

12. Quick Revision / Viva Questions

- What is DevOps? — A culture and set of practices that integrates development and operations to deliver software continuously and reliably.
- What is CI? — Automatically building and testing code changes when they are integrated.
- What is CD? — Automating delivery/release of validated software to environments.
- Git vs GitHub? — Git is version-control software; GitHub is a platform for hosting and collaborating on Git repositories.
- Image vs container? — An image is a packaged template; a container is a running instance of that image.

- Docker vs Kubernetes? — Docker is primarily used for containerization; Kubernetes manages and orchestrates containers at scale.
- Why monitoring? — To observe health, performance, availability, and failures.
- What is IaC? — Managing infrastructure through machine-readable configuration files.